Chapter 3. Tool 만들기와 Tool Calling

학습 목표

이번 장에서는 Python 함수를 Tool로 등록하고, AI Agent가 이를 자동으로 호출하는 과정을 학습합니다.

학습이 끝나면 다음 내용을 수행할 수 있습니다.

- Tool의 개념 이해
- Python 함수 작성
- Tool 등록
- Tool Calling 동작 원리 이해
- Agent와 Tool 연동

이 장의 용어

용어	영문 · 약어 풀이	설명
Tool	도구	Agent가 호출할 수 있는 기능. Google ADK에서는 Python 함수 하나가 Tool이 됨
Tool Calling	Function Calling, 도구(함수) 호출	LLM이 질문을 보고 필요한 Tool을 골라 실행을 요청하는 기능. 다른 프레임워크에서는 Function Calling이라고 부름
함수	Function	입력을 받아 정해진 작업을 하고 결과를 돌려주는 코드 묶음. `def 이름(...):`으로 정의
매개변수 / 인자	Parameter / Argument	매개변수는 함수가 받기로 정한 입력 이름(`sides`), 인자는 실제로 넘겨진 값(`20`)
반환값	Return value	함수가 `return`으로 돌려주는 결과. Tool의 반환값이 LLM에게 전달됨
타입 힌트	Type Hint	`sides: int`, `-> str`처럼 입력과 반환값의 자료형을 적는 표기. LLM이 Tool의 입출력 형식을 이해하는 데 사용

용어	영문 · 약어 풀이	설명
Docstring	Documentation String, 문서화 문자열	함수 바로 아래 따옴표 세 개(`"""`) 사이에 적는 설명문. LLM이 Tool을 고를 때 읽는 사용 설명서
메타데이터	Metadata	데이터를 설명하는 데이터. Tool의 이름·Docstring·매개변수·반환 타입이 Tool의 메타데이터
모듈 / 표준 라이브러리	Module / Standard Library	모듈은 기능을 모아 둔 Python 파일, 표준 라이브러리는 Python에 기본 포함된 모듈 모음(datetime, random 등). `import`로 가져옴
딕셔너리	Dictionary, dict	`{"USD": "1,385원"}`처럼 이름(키)으로 값을 찾는 Python 자료형. JSON과 구조가 같음
eval	evaluate	문자열을 Python 코드로 그대로 실행하는 함수. 간편하지만 보안에 취약해 실습용으로만 사용
Events 탭	—	adk web에서 LLM과 Tool 사이의 호출(functionCall)과 응답(functionResponse)을 순서대로 보여주는 화면

1. Tool이란?

앞에서 우리는 Google ADK를 이용하여 첫 번째 AI Agent를 만들어 보았습니다.

Agent는 사용자와 자연스럽게 대화할 수 있었지만 몇 가지 한계가 있었습니다.

예를 들어 다음과 같은 질문을 해보겠습니다.

> 현재 시간을 알려주세요.

또는

> 서울의 현재 날씨를 알려주세요.

또는

> 제 이메일로 회의록을 보내주세요.

이러한 요청은 모두 실제 업무에서 자주 사용하는 기능입니다.

하지만 지금까지 만든 Agent는 이러한 작업을 수행할 수 없습니다.

왜일까요?

현재 Agent는 **LLM만 사용할 수 있기 때문**입니다.

LLM은 언어를 이해하고 문장을 생성하는 능력은 뛰어나지만, 컴퓨터의 기능을 직접 사용할 수는 없습니다.

이러한 한계를 해결하기 위해 사용하는 것이 **Tool**입니다.

> **Tool은 AI Agent가 외부 기능이나 시스템을 사용할 수 있도록 연결해 주는 기능입니다.**

쉽게 말하면 AI가 사용하는 **도구(Tool)** 라고 생각하면 됩니다.

1.1. 사람도 Tool을 사용한다

Tool의 개념은 사람의 행동과 매우 비슷합니다.

예를 들어 우리가 일을 할 때를 생각해 보겠습니다.

계산이 필요한 경우

사람은 계산기를 사용합니다.

```
사람

↓

계산기

↓

결과
```

문서를 작성하는 경우

컴퓨터를 사용합니다.

```
사람

↓

Word

↓

문서 작성
```

길을 찾는 경우

지도 애플리케이션을 사용합니다.

```
사람

↓

지도 앱

↓

길 안내
```

사람은 상황에 따라 필요한 도구를 선택합니다.

AI도 마찬가지입니다.

필요한 작업이 있다면 적절한 Tool을 사용합니다.

1.2. Tool이 필요한 이유

LLM은 많은 지식을 가지고 있지만 모든 일을 할 수 있는 것은 아닙니다.

예를 들어 다음 질문을 살펴보겠습니다.

현재 시간

```
지금 몇 시인가요?
```

현재 시간을 알려면 컴퓨터의 시간을 확인해야 합니다.

LLM은 현재 시간을 알지 못합니다.

현재 날씨

```
오늘 서울 날씨 알려줘.
```

실시간 날씨는 Weather API를 조회해야 합니다.

LLM은 인터넷을 검색하는 기능이 기본적으로 없습니다.

데이터 조회

```
홍길동 학생 정보를 조회해 주세요.
```

학생 정보는 데이터베이스에 저장되어 있습니다.

LLM은 회사의 데이터베이스를 직접 읽을 수 없습니다.

이메일 발송

```
회의록을 팀원들에게 보내주세요.
```

메일을 보내려면 Gmail API를 사용해야 합니다.

LLM은 이메일을 직접 발송할 수 없습니다.

즉,

LLM은 생각할 수는 있지만 행동할 수는 없습니다.

행동을 가능하게 해주는 것이 Tool입니다.

1.3. Tool이 추가되면 무엇이 달라질까?

Tool이 없는 Agent는 다음과 같이 동작합니다.

```
사용자

↓

Agent

↓

Gemini

↓

응답
```

Gemini는 답변만 생성합니다.

Tool이 추가되면 구조가 다음과 같이 변경됩니다.

```
사용자
↓
Agent
↓
Gemini
↓
Tool
↓
결과
↓
Gemini
↓
응답
```

Gemini는 필요할 경우 Tool을 실행하고, Tool의 결과를 이용하여 최종 답변을 생성합니다.

즉, AI는 단순히 대화하는 것을 넘어 **실제 작업을 수행하는 Agent**가 됩니다.

1.4. Tool은 특별한 프로그램일까?

많은 사람들이 Tool은 특별한 형태로 만들어야 한다고 생각합니다.

하지만 Google ADK에서는 매우 간단합니다.

Python 함수 하나가 Tool이 될 수 있습니다.

예를 들어

```python
def get_current_time():
    return "2026-07-03 10:30"
```

이 함수는 일반적인 Python 함수입니다.

Google ADK에서는 이 함수를 Agent에 등록하면 Tool이 됩니다.

즉,

```
Python 함수

↓

Google ADK 등록

↓

Tool
```

복잡한 클래스를 만들 필요도 없고 특별한 문법도 필요하지 않습니다.

이것이 Google ADK의 가장 큰 장점 중 하나입니다.

1.5. Tool은 어떤 일을 할 수 있을까?

Tool은 Python으로 할 수 있는 거의 모든 작업을 수행할 수 있습니다.

예를 들어 다음과 같은 기능을 구현할 수 있습니다.

Tool	수행 기능
Time Tool	현재 시간 조회
Calculator Tool	계산 수행
Weather Tool	날씨 조회
Database Tool	데이터베이스 조회
Gmail Tool	이메일 발송
File Tool	파일 읽기 및 저장
Search Tool	인터넷 검색
GitHub Tool	저장소 조회
Slack Tool	메시지 전송

이처럼 Tool을 추가할수록 AI Agent의 능력도 확장됩니다.

1.6. Tool은 AI에게 새로운 능력을 부여한다

사람이 새로운 자격증을 취득하면 할 수 있는 일이 늘어나는 것처럼, AI도 새로운 Tool을 추가하면 수행할 수 있는 업무가 늘어납니다.

초기 Agent

```
질문
↓
답변
```

Time Tool 추가

```
질문
↓
현재 시간 조회
↓
답변
```

Database Tool 추가

```
질문
↓
학생 정보 조회
↓
답변
```

Gmail Tool 추가

```
질문
↓
메일 발송
↓
완료
```

Tool이 많아질수록 Agent는 더 다양한 업무를 처리할 수 있습니다.

1.7. Tool과 API의 관계

Tool은 API를 사용할 수도 있고, 단순한 Python 코드일 수도 있습니다.

단순 Python Tool

```
랜덤 숫자 생성

↓

Python random 라이브러리 사용
```

API 기반 Tool

```
날씨 조회

↓

Weather API 호출
```

데이터베이스 Tool

```
학생 조회

↓

MySQL 조회
```

즉, Tool은 **외부 시스템과 AI를 연결하는 연결 지점(Bridge)** 역할을 합니다.

1.8. 실무에서 Tool은 어떻게 사용될까?

AI Agent는 대부분 여러 Tool을 함께 사용합니다.

예를 들어 교육 관리 시스템에서는 다음과 같은 Tool을 사용할 수 있습니다.

```
학생 정보 조회

↓
```

```
Database Tool
```

```
출석 현황 조회

↓

Attendance Tool
```

```
공지 메일 발송

↓

Gmail Tool
```

```
학습 보고서 생성

↓

PDF Tool
```

사용자는 단순히

> "홍길동 학생의 출석률을 확인하고 보고서를 이메일로 보내줘."

라고 요청하면, Agent는 필요한 Tool을 자동으로 선택하여 작업을 수행합니다.

정리

이번 절에서는 AI Agent의 핵심 요소인 Tool의 개념을 학습했습니다.

개념	설명
Tool	AI가 사용할 수 있는 외부 기능 또는 시스템
역할	AI에게 새로운 능력을 제공
구현	일반 Python 함수를 Tool로 등록
활용	API 호출, DB 조회, 파일 처리, 메일 발송 등
특징	LLM이 직접 수행할 수 없는 작업을 처리

---

2. 첫 번째 Tool 만들기

2.1. Tool 파일 생성

앞 절에서는 Tool이 AI Agent에게 새로운 능력을 제공하는 기능이라는 것을 학습했습니다.

이번에는 직접 Tool을 만들어 보겠습니다.

Google ADK에서는 특별한 문법을 배우지 않아도 됩니다.

일반 Python 함수 하나가 Tool이 됩니다.

이번 실습에서는 가장 간단한 **현재 시간을 알려주는 Tool**을 만들어 보겠습니다.

프로젝트 구조는 다음과 같습니다.

```
my-adk/
│
├── agent/
│   └── agent.py
│
└── tools/
    └── time_tool.py
```

2.2. 첫 번째 Tool 작성

다음과 같이 Python 함수를 작성합니다.

`tools/time_tool.py`

```python
from datetime import datetime

def get_current_time() -> str:
    """
    현재 시간을 반환합니다.
    """

    now = datetime.now()

    return now.strftime("%Y-%m-%d %H:%M:%S")
```

이 함수는 현재 컴퓨터의 날짜와 시간을 문자열 형태로 반환합니다.

아직은 Google ADK와 연결되지 않은 **일반 Python 함수**입니다.

2.3. 코드 살펴보기

먼저 `datetime` 모듈을 가져옵니다.

```
from datetime import datetime
```

`datetime`은 현재 날짜와 시간을 다루는 Python 표준 라이브러리(Standard Library, Python을 설치하면 함께 들어 있어 별도 설치 없이 쓰는 기능 모음)입니다. `from ... import ...`는 그 안의 기능을 현재 파일로 가져오는 문장입니다. 함수 이름 뒤의 `-> str`은 "이 함수는 문자열(str)을 반환한다"는 타입 힌트(Type Hint)로, LLM이 Tool의 결과 형식을 이해하는 데 쓰입니다.

현재 시간을 가져옵니다.

```
now = datetime.now()
```

마지막으로

```
return now.strftime("%Y-%m-%d %H:%M:%S")
```

을 이용하여 사람이 읽기 쉬운 형태로 변환합니다.

예를 들어

```
2026-07-03 14:35:20
```

을 반환합니다.

2.4. Agent에 Tool 등록하기

이제 작성한 함수를 Agent에 연결해 보겠습니다.

`agent/agent.py`

```
from google.adk.agents import Agent
from tools.time_tool import get_current_time

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="hello_agent",
```

```
    description="첫 번째 AI Agent",
    instruction="""
    당신은 친절한 AI 비서입니다.

    사용자의 질문에 이해하기 쉽게 답변하세요.

    현재 시간이 필요하면
    get_current_time Tool을 사용하세요.
    """,
    tools=[
        get_current_time
    ]
)
```

여기서 중요한 부분은

```
tools=[
    get_current_time
]
```

입니다.

이 코드를 통해 Google ADK는 `get_current_time()` 함수를 Tool로 등록합니다.

Tool 등록 과정

Tool이 등록되면 내부적으로 다음과 같은 구조가 만들어집니다.

```
Agent
|
├── Gemini
└── Tool
      └── get_current_time()
```

이제 Gemini는 필요할 경우 `get_current_time()` 함수를 호출할 수 있습니다.

2.5. Tool 실행

프로젝트 루트에서 Agent를 실행합니다.

```
adk web
```

또는

```
adk run agent
```

실행이 완료되면 다음 질문을 입력합니다.

```
현재 시간을 알려줘.
```

Google ADK는 내부적으로 다음과 같이 동작합니다.

```
사용자

↓

Agent

↓

Gemini

↓

Tool 필요 판단

↓

get_current_time()

↓

현재 시간 반환

↓

Gemini

↓

응답 생성

↓

사용자
```

2.6. 실행 결과

예를 들어 다음과 같은 응답을 받을 수 있습니다.

```
현재 시간은

2026-07-03 14:35:20

입니다.
```

중요한 점은 개발자가

```
get_current_time()
```

를 직접 호출하지 않았다는 것입니다.

Google ADK가 자동으로 Tool을 실행한 것입니다.

Tool 호출 확인하기

Tool이 실제로 호출되었는지는 `adk web` 화면에서 직접 확인할 수 있습니다.

1. 질문을 입력한 뒤 화면 왼쪽의 **Events** 탭을 엽니다.
2. 이벤트 목록에서 `functionCall` 항목을 클릭하면 호출된 Tool 이름(`get_current_time`)과 전달된 인자(Arguments)가 표시됩니다.
3. 이어지는 `functionResponse` 항목에는 Tool이 반환한 값이 표시됩니다.

> **참고**
> 이후 모든 실습에서 "Tool이 호출됐는지"를 판단할 때 이 Events 탭을 사용합니다. 응답 문장만 보고 판단하면 LLM이 Tool 없이 지어낸 답과 구분할 수 없습니다.

2.7. Tool은 일반 함수와 무엇이 다를까?

많은 사람들이 Tool을 특별한 객체라고 생각합니다.

하지만 실제로는 그렇지 않습니다.

다음 두 코드를 비교해 보겠습니다.

일반 Python 함수

```
def get_current_time():
    ...
```

Google ADK Tool

```
tools=[
    get_current_time
]
```

차이는 단 하나입니다.

Agent에 등록했는지 여부입니다.

즉,

```
Python 함수

↓

Agent 등록

↓

Tool
```

Google ADK는 등록된 함수를 AI가 사용할 수 있도록 관리합니다.

Tool을 여러 개 등록할 수도 있다

현재는 하나의 Tool만 등록했지만 여러 개의 Tool도 등록할 수 있습니다.

```
tools=[
    get_current_time,
    get_weather,
    calculate,
]
```

처럼 작성하면 Gemini는 질문에 따라 필요한 Tool을 자동으로 선택합니다.

정리

이번 절에서는 Google ADK에서 첫 번째 Tool을 직접 만들어 보았습니다.

구성 요소	역할
Python 함수	Tool의 기본 형태
`tools`	Agent가 사용할 Tool 목록
Tool 등록	Python 함수를 Agent에 연결
Tool 실행	Gemini가 필요할 때 자동 호출
Events 탭	Tool 호출 여부, 인자, 반환값 확인

3. Tool Calling 이해 및 실습

3.1. Tool Calling이란?

앞 절에서는 현재 시간을 반환하는 Tool을 만들고 Agent에 등록했습니다.

이제 사용자가 다음과 같이 질문한다고 가정해 보겠습니다.

```
현재 시간을 알려줘.
```

이 질문을 받은 AI는 어떻게 현재 시간을 알려줄 수 있을까요?

Google ADK는 내부적으로 **Tool Calling**이라는 기능을 수행합니다.

Tool Calling이란,

> **AI가 사용자의 요청을 분석하여 필요한 Tool을 자동으로 선택하고 실행하는 기능**입니다.

중요한 점은 **개발자가 직접 Tool을 호출하지 않는다는 것**입니다.

예를 들어 일반 Python 프로그램에서는

```python
result = get_current_time()
print(result)
```

처럼 함수를 직접 호출합니다.

반면 Google ADK에서는 사용자가 질문만 입력하면 AI가 스스로 Tool을 실행합니다.

3.2. 일반 프로그램과 AI Agent 비교

기존 프로그램은 개발자가 실행 순서를 모두 작성해야 합니다.

```
사용자 입력
↓
if 문
↓
get_current_time()
↓
출력
```

반면 AI Agent에서는

```
사용자 질문
↓
LLM
↓
Tool 선택
↓
Tool 실행
↓
응답 생성
```

개발자는 Tool만 등록하면 되고, 언제 호출할지는 LLM이 판단합니다.

3.3. Tool Calling 동작 과정

예를 들어 사용자가 다음과 같이 질문합니다.

```
현재 시간을 알려줘.
```

Google ADK는 다음 순서로 동작합니다.

```
① 사용자 질문
↓
② Agent
↓
③ Gemini가 질문 분석
↓
④ Tool 목록 확인
↓
⑤ get_current_time 선택
↓
⑥ Python 함수 실행
↓
⑦ 결과 반환
↓
⑧ Gemini가 자연어 응답 생성
↓
⑨ 사용자
```

겉으로 보기에는 한 번의 질문과 답변처럼 보이지만, 내부에서는 여러 단계의 처리가 이루어집니다.

3.4. Tool은 어떻게 선택될까?

많은 사람들이

> "함수 이름이 `get_current_time`이라서 선택되는 것 아닌가요?"

라고 생각합니다.

하지만 실제로는 그렇지 않습니다.

Google ADK는 Tool의 **메타데이터(Metadata, 함수를 설명하는 정보)**를 LLM에 제공합니다.

대표적으로 다음 정보를 활용합니다.

- 함수 이름
- 함수 설명(Docstring)
- 매개변수(Parameter)
- 반환 타입(Return Type)

그중에서도 가장 중요한 것은 **Docstring**입니다.

3.5. Docstring의 역할

다음 Tool을 살펴보겠습니다.

```
def get_current_time() -> str:
    """
    현재 날짜와 시간을 반환합니다.
    """
```

여기서

```
"""
현재 날짜와 시간을 반환합니다.
"""
```

부분이 Docstring(Documentation String, 함수 바로 아래 따옴표 세 개 사이에 적는 설명문)입니다.

이 문장은 사람이 읽기 위한 주석이 아니라, LLM에게

> "이 Tool은 어떤 기능을 하는가?"

를 설명하는 역할을 합니다.

즉, Docstring은 Tool의 사용 설명서입니다.

3.6. Docstring이 중요한 이유

예를 들어 다음 두 Tool이 있다고 가정해 보겠습니다.

```
def tool1():
    """
```

```
    현재 시간을 반환합니다.
    """
```

```
def tool2():
    """
    현재 서울의 날씨를 조회합니다.
    """
```

사용자가

```
지금 몇 시인가요?
```

라고 질문하면 LLM은 Tool 설명을 읽고 `tool1()`이 적절하다고 판단합니다.

반대로

```
서울 날씨 알려줘.
```

라고 질문하면 `tool2()`를 선택합니다.

즉, LLM은 **질문의 의미와 Tool 설명을 비교하여 가장 적절한 Tool을 선택**합니다.

3.7. 다른 질문도 해보기

다음과 같은 질문도 입력해 봅니다.

```
지금 몇 시예요?
```

```
현재 시간이 궁금해요.
```

```
오늘 날짜도 알려주세요.
```

표현이 달라져도 AI는 같은 Tool을 사용할 수 있습니다.

이는 단순히 키워드를 비교하는 것이 아니라, 질문의 의미를 이해하여 Tool을 선택하기 때문입니다.

3.8. Tool이 호출되지 않는 경우

모든 질문에서 Tool이 실행되는 것은 아닙니다.

예를 들어

```
파이썬이 무엇인가요?
```

와 같은 질문은 Tool이 필요하지 않습니다.

이 경우 Gemini는 자체 지식을 이용하여 응답을 생성합니다.

즉, Tool은 **필요한 경우에만** 호출됩니다.

Events 탭을 열어 보면 이 질문에는 `functionCall` 이벤트가 생기지 않는 것을 확인할 수 있습니다.

Tool Calling의 장점

Tool Calling을 사용하면 다음과 같은 장점이 있습니다.

- 개발자가 실행 순서를 직접 작성할 필요가 없습니다.
- 자연어만으로 다양한 기능을 사용할 수 있습니다.
- 여러 Tool을 하나의 Agent에서 관리할 수 있습니다.
- Tool을 추가할수록 Agent의 기능을 쉽게 확장할 수 있습니다.

정리

이번 절에서는 Google ADK의 핵심 기능인 Tool Calling을 학습했습니다.

개념	설명
Tool Calling	AI가 필요한 Tool을 자동으로 선택하고 실행하는 기능
Tool 선택 기준	함수 설명(Docstring), 매개변수, 반환 타입 등 메타데이터
Docstring	Tool의 기능을 LLM에 설명하는 역할
실행 과정	질문 분석 → Tool 선택 → 실행 → 응답 생성

---

4. 여러 Tool을 동시에 등록

4.1. 하나의 Tool만으로는 부족하다

앞 절에서는 현재 시간을 알려주는 Tool을 만들어 보았습니다.

하지만 실제 AI Agent는 하나의 기능만 수행하지 않습니다.

예를 들어 다음과 같은 요청을 생각해 보겠습니다.

- 현재 시간을 알려줘.
- 125 × 36을 계산해줘.
- 주사위를 굴려줘.

이 세 가지 작업은 모두 서로 다른 기능이며, 모두 **LLM이 혼자서는 정확히 수행할 수 없는 일**입니다.

- 현재 시간: LLM은 지금이 몇 시인지 모릅니다.
- 계산: LLM은 큰 수의 곱셈에서 자주 틀립니다.
- 주사위: LLM은 진짜 난수를 만들 수 없습니다. "무작위"처럼 보이는 숫자를 지어낼 뿐입니다.

따라서 하나의 Tool만으로는 처리할 수 없습니다.

AI Agent는 여러 Tool을 함께 등록하고, 상황에 맞는 Tool을 선택하여 사용합니다.

4.2. 새로운 Tool 만들기

이번 실습에서는 기존의 `get_current_time()` Tool 외에 두 개의 Tool을 추가합니다.

프로젝트 구조는 다음과 같습니다.

```
my-adk/
│
├── agent/
│   └── agent.py
│
└── tools/
    ├── time_tool.py
    ├── calculator_tool.py
    └── dice_tool.py
```

Calculator Tool

`tools/calculator_tool.py`

```python
def calculate(expression: str) -> str:
    """
```

```
    간단한 수식을 계산합니다.
    예: "125 * 36", "(3 + 5) / 2"
    """

    try:
        return str(eval(expression))
    except Exception:
        return "계산할 수 없습니다."
```

> **주의**
> `eval()`은 전달된 문자열을 Python 코드로 그대로 실행하므로 보안에 취약합니다. 실습용 예제로만 사용하고, 실제 서비스에서는 수식 파서 라이브러리 등 안전한 방법을 사용해야 합니다.

Dice Tool

`tools/dice_tool.py`

```python
import random

def roll_dice(sides: int = 6) -> int:
    """
    주사위를 굴려 1부터 sides 사이의 숫자 하나를 무작위로 반환합니다.
    sides는 주사위 면의 개수이며, 지정하지 않으면 6면 주사위를 사용합니다.
    """

    return random.randint(1, sides)
```

이 Tool은 외부 API 없이 Python 표준 라이브러리만 사용합니다. 1.7절에서 본 "단순 Python Tool"의 예입니다.

매개변수 `sides`에 기본값이 있으므로, 사용자가 "20면 주사위를 굴려줘"라고 말하면 LLM이 `sides=20`을 채워 호출하고, 그냥 "주사위 굴려줘"라고 하면 기본값 6이 사용됩니다.

4.3. 여러 Tool 등록하기

이제 세 개의 Tool을 Agent에 등록합니다.

`agent/agent.py`

```python
from google.adk.agents import Agent

from tools.time_tool import get_current_time
from tools.calculator_tool import calculate
from tools.dice_tool import roll_dice

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="multi_tool_agent",
    description="여러 Tool을 사용하는 Agent",
    instruction="""
    사용자의 질문에 적절한 Tool을 선택하여 답변하세요.
    """,
    tools=[
        get_current_time,
        calculate,
        roll_dice
    ]
)
```

여기서 `tools` 목록에 세 개의 함수가 등록되었습니다.

```python
tools=[
    get_current_time,
    calculate,
    roll_dice
]
```

이제 Agent는 세 가지 기능을 모두 사용할 수 있습니다.

4.4. AI는 어떤 Tool을 선택할까?

사용자가 질문을 입력하면 Gemini는 등록된 Tool을 확인하고 가장 적절한 Tool을 선택합니다.

예를 들어 다음과 같은 질문을 입력해 보겠습니다.

현재 시간

```
현재 시간을 알려줘.
```

내부 동작

```
사용자

↓

Gemini

↓

get_current_time()

↓

응답
```

계산

```
125 * 36 계산해줘.
```

내부 동작

```
사용자

↓

Gemini

↓

calculate()

↓

응답
```

주사위

```
주사위 굴려줘.
```

내부 동작

```
사용자

↓

Gemini

↓

roll_dice()

↓

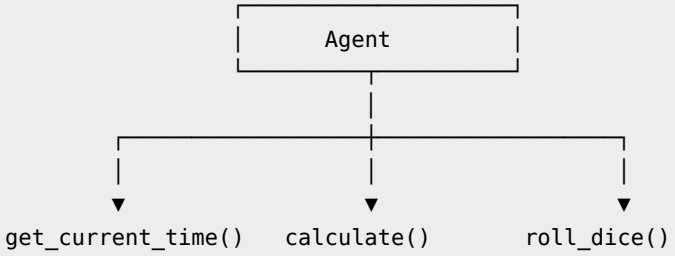
응답
```

같은 질문을 여러 번 입력해 보세요. Tool이 호출되므로 매번 다른 숫자가 나옵니다. Events 탭에서 `roll_dice`가 실제로 호출되었는지, `sides` 인자에 어떤 값이 들어갔는지 확인합니다.

4.5. 하나의 Agent, 여러 기능

지금까지는 Tool 하나당 Agent 하나처럼 보였지만 실제로는 하나의 Agent가 여러 Tool을 사용할 수 있습니다.

구조를 그림으로 표현하면 다음과 같습니다.



Gemini는 사용자의 요청을 분석한 후 가장 적절한 Tool을 선택합니다.

4.6. Tool 선택은 자동으로 이루어진다

중요한 점은 개발자가 다음과 같은 코드를 작성하지 않는다는 것입니다.

```python
if "시간" in question:
    get_current_time()
```

```
elif "계산" in question:
    calculate()
```

이러한 조건문은 Google ADK에서 필요하지 않습니다.

대신 Gemini가 질문의 의미를 이해하여 적절한 Tool을 선택합니다.

이것이 Tool Calling의 가장 큰 장점입니다.

4.7. Tool은 계속 추가할 수 있다

프로젝트가 커질수록 Tool도 함께 늘어납니다.

예를 들어 교육 관리 시스템이라면 다음과 같은 Tool을 추가할 수 있습니다.

```
tools/

├── attendance_tool.py
├── student_tool.py
├── score_tool.py
├── report_tool.py
├── mail_tool.py
└── calendar_tool.py
```

그리고 Agent에는 필요한 Tool만 등록합니다.

```
tools=[
    search_student,
    attendance,
    send_mail,
    create_report
]
```

이처럼 Tool을 추가하는 것만으로 Agent의 기능을 계속 확장할 수 있습니다.

실습 과제

실습 3-1. 나만의 Tool 추가 (심화)

자신만의 Tool을 하나 만들어 Agent에 등록해 보세요.

아래는 예시입니다. 환율 조회 Tool은 **미리 정해 둔 고정값을 반환하는 가짜 데이터**이며, 실제 환율을 조회하는 Tool은 Chapter 7의 실습 7-2에서 만듭니다.

예시 1. 환율 조회 (고정값)

`tools/exchange_rate_tool.py`

```
def get_exchange_rate(currency: str) -> str:
    """
    지정한 통화의 원화 환율을 반환합니다. (실습용 고정값)
    지원 통화: USD, JPY, EUR
    """

    rates = {
        "USD": "1 USD = 1,385원",
        "JPY": "100 JPY = 940원",
        "EUR": "1 EUR = 1,620원",
    }

    return rates.get(currency.upper(), "지원하지 않는 통화입니다.")
```

`agent/agent.py`

```
from google.adk.agents import Agent

from tools.time_tool import get_current_time
from tools.calculator_tool import calculate
from tools.dice_tool import roll_dice
from tools.exchange_rate_tool import get_exchange_rate

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="multi_tool_agent",
    description="여러 Tool을 사용하는 Agent",
    instruction="""
    사용자의 질문에 적절한 Tool을 선택하여 답변하세요.
    """,
    tools=[
        get_current_time,
```

```
        calculate,
        roll_dice,
        get_exchange_rate
    ]
)
```

예시 2. 메모 저장

tools/memo_tool.py

```python
from pathlib import Path

MEMO_FILE = Path("memo.txt")


def save_memo(content: str) -> str:
    """
    사용자가 전달한 메모를 파일에 저장합니다.

    Args:
        content: 저장할 메모 내용

    Returns:
        저장 결과 메시지
    """

    with open(MEMO_FILE, "a", encoding="utf-8") as f:
        f.write(content + "\n")

    return f"메모를 저장했습니다.\n내용: {content}"
```

agent/agent.py

```python
from google.adk.agents import Agent

from tools.time_tool import get_current_time
from tools.calculator_tool import calculate
from tools.dice_tool import roll_dice
from tools.exchange_rate_tool import get_exchange_rate
```

```python
from tools.memo_tool import save_memo

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="multi_tool_agent",
    description="여러 Tool을 사용하는 Agent",
    instruction="""
    사용자의 질문에 적절한 Tool을 선택하여 답변하세요.
    """,
    tools=[
        get_current_time,
        calculate,
        roll_dice,
        get_exchange_rate,
        save_memo
    ]
)
```

예시 3. 메모 조회

`tools/memo_tool.py`에 함수를 추가합니다.

```python
def read_memo() -> str:
    """
    저장된 메모를 조회합니다.
    """

    if not MEMO_FILE.exists():
        return "저장된 메모가 없습니다."

    content = MEMO_FILE.read_text(encoding="utf-8")

    if not content.strip():
        return "저장된 메모가 없습니다."

    return f"저장된 메모 목록입니다.\n{content}"
```

`agent/agent.py`

```python
from google.adk.agents import Agent

from tools.time_tool import get_current_time
from tools.calculator_tool import calculate
from tools.dice_tool import roll_dice
from tools.exchange_rate_tool import get_exchange_rate
from tools.memo_tool import save_memo, read_memo

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="multi_tool_agent",
    description="여러 Tool을 사용하는 Agent",
    instruction="""
    사용자의 질문에 적절한 Tool을 선택하여 답변하세요.
    """,
    tools=[
        get_current_time,
        calculate,
        roll_dice,
        get_exchange_rate,
        save_memo,
        read_memo
    ]
)
```

"오늘 회의 3시로 변경이라고 메모해줘" → "저장된 메모 보여줘" 순서로 질문해 두 Tool이 각각 호출되는지 Events 탭에서 확인합니다.

정리

이번 절에서는 여러 Tool을 하나의 Agent에 등록하고, AI가 상황에 맞는 Tool을 자동으로 선택하는 과정을 실습했습니다.

구성 요소	역할
`tools`	Agent가 사용할 Tool 목록
여러 Tool 등록	하나의 Agent에 다양한 기능 추가
Tool 선택	Gemini가 질문을 분석하여 자동 선택

구성 요소	역할
매개변수 채우기	LLM이 질문에서 인자 값을 추출해 전달 (예: `sides=20`)
확장성	Tool을 추가할수록 Agent의 기능 확장

체크포인트 질문

1. Google ADK에서 Python 함수를 Tool로 만들기 위해 필요한 작업은 무엇인가요? → **Agent의 `tools` 목록에 함수를 등록하는 것. 별도의 클래스나 문법은 필요 없음**
2. 사용자가 "지금 몇 시예요?"라고 물었을 때 LLM이 `get_current_time`을 선택하는 근거는 함수 이름일까요, Docstring일까요? → **Docstring(함수 설명)이 가장 중요하며, 함수 이름·매개변수·반환 타입도 함께 참고함**
3. "파이썬이 무엇인가요?"라는 질문에는 Tool이 호출되나요? → **호출되지 않음. Tool은 LLM이 필요하다고 판단한 경우에만 호출됨**
4. 주사위 굴리기를 LLM에게 직접 시키지 않고 Tool로 만든 이유는 무엇인가요? → **LLM은 진짜 난수를 만들 수 없기 때문. 현재 시간·정확한 계산처럼 LLM이 스스로 할 수 없는 일이 Tool의 대상**
5. Tool을 세 개 등록했을 때, 어떤 Tool을 쓸지 정하는 `if` 문을 개발자가 작성해야 하나요? → **아니오. LLM이 질문의 의미와 Tool 설명을 비교해 자동으로 선택함**
6. `adk web`에서 Tool이 실제로 호출되었는지 확인하려면 어디를 보면 되나요? → **Events 탭의 `functionCall`(Tool 이름, 인자)과 `functionResponse`(반환값)**